

EXPERIMENT 4

Creating Requirement Files — Managing Dependencies with requirements.txt

Aim

To create isolated Python virtual environments and manage project dependencies reproducibly using requirements.txt.

Theory

Python projects depend on third-party packages that are frequently updated. Without isolation, installing packages globally causes version conflicts between projects ('dependency hell'). A virtual environment is an isolated directory containing its own Python interpreter and site-packages, so each project can pin its own versions.

requirements.txt is a plain-text manifest listing each dependency and, ideally, its exact pinned version (e.g. numpy==1.26.4). It lets any developer or CI server recreate the exact environment with a single command, guaranteeing reproducibility across machines.

The workflow is: create a venv, activate it, install packages, and freeze the resolved versions into requirements.txt. Version specifiers (==, >=, ~=) control how strictly versions are matched. Tools such as pip-tools or Poetry offer more advanced dependency resolution.

Software / Tools Required

Python 3.10+, pip, venv module.

Procedure (Step-by-Step)

Step 1. Create a virtual environment inside the project folder.

```
python -m venv .venv
```

Step 2. Activate the environment (choose the line for your OS).

```
# Linux / macOS:
source .venv/bin/activate
# Windows (PowerShell):
.venv\Scripts\Activate.ps1
```

Step 3. Confirm the environment is active and pip is isolated.

```
which python  # points inside .venv
pip list      # near-empty list
```

Step 4. Install the packages your project needs.

```
pip install numpy pandas matplotlib flask
```

Step 5. Freeze the exact installed versions into requirements.txt.

```
pip freeze > requirements.txt
cat requirements.txt
```

Step 6. Alternatively, hand-write a curated requirements file with version specifiers.

```
# requirements.txt
numpy==1.26.4
pandas>=2.0,<3.0
matplotlib~=3.8
flask==3.0.3
```

Step 7. Recreate the environment elsewhere to test reproducibility.

```
deactivate
python -m venv .venv2 && source .venv2/bin/activate
pip install -r requirements.txt
```

Step 8. Separate development-only dependencies into a second file.

```
# requirements-dev.txt
-r requirements.txt
pytest
black
flake8
```

Step 9. Add the venv folder to .gitignore so it is never committed.

```
echo ".venv/\n.venv2/" >> .gitignore
```

Step 10. Check for outdated packages when maintaining the project.

```
pip list --outdated
```

Expected Output

requirements.txt lists every dependency with a pinned version. Running 'pip install -r requirements.txt' in a fresh environment reinstalls the identical set of packages, confirming reproducibility.

Conclusion

Virtual environments and requirements.txt were used to isolate and reproducibly manage project dependencies — a mandatory practice for any collaborative or deployable Python/ML project.

Viva Questions

1. Why should the .venv directory be excluded from version control?
2. What is the difference between '==', '>=' and '~=' version specifiers?
3. What does 'pip freeze' do and how does it differ from 'pip list'?
4. Why separate requirements.txt from requirements-dev.txt?
5. What problem does a virtual environment solve?